

Implementation Backlog

Epic 1: LangGraph foundation

Work items

- Create a graph package with builder, state, and node modules.
- Define GraphState, VAFinding, and remediation candidate models.
- Add a graph runner service that can be invoked from the webhook layer.

Acceptance criteria

- A stored webhook event can be converted into graph input.
- The graph can execute all four nodes with stub implementations.

Epic 2: Node 01 vulnerability extraction

Work items

- Implement Trivy console parser.
- Support multi-row package entries and repeated CVEs.
- Emit normalized issue list plus general remediation hints.

Acceptance criteria

- The existing sample payload yields a non-empty structured issue list.
- CVE ids, package names, installed versions, fixed versions, and targets are preserved.

Epic 3: Node 02 manifest fetch and BOM-friendly planning

Work items

- Wrap current GitHub fetch logic in repo-reader service.
- Parse Maven parent/BOM/property management.
- Parse Gradle platform/property/version ownership.
- Parse Dockerfile base image and package upgrade points.
- Generate candidate changes rather than direct file patches.

Acceptance criteria

- The node can explain why a change belongs in a BOM, property, direct dependency, or Dockerfile.
- The node can produce more than one candidate when several strategies are possible.

Epic 4: Node 03 validation gate

Work items

- Detect BOM-owned dependencies.
- Block inline versions that break manifest ownership.
- Resolve duplicate or conflicting candidate plans.
- Score confidence and manual-review flags.

Acceptance criteria

- Invalid direct bumps are rejected.
- A validated plan includes rationale and complete file write set.

Epic 5: Node 04 GitHub branch push

Work items

- Add multi-file write support.

- Support branch create-or-reuse behavior.
- Create PR or compare URL.
- Persist final run output.

Acceptance criteria

- One validated change plan can update all required files in one branch.
- Existing branch reuse works safely.

Epic 6: API and UI integration

Work items

- Add run status endpoints.
- Show graph progress and node outputs in UI.
- Preserve current tester flow while the AI flow matures.

Acceptance criteria

- Operators can see whether the graph is waiting, running, validated, blocked, or pushed.

Cross-cutting risks

- Current repo write logic is single-file only.
- Current event model mixes raw event data and remediation outputs in one object.
- Large console logs can make prompt size expensive if not pre-parsed before model use.
- Gradle and Maven ownership logic can produce false positives if parsers are too shallow.

Recommended build order

1. Graph scaffolding
2. Node 01 parser
3. Node 02 repo reader + manifest parsing
4. Node 03 validation

5. Node 04 multi-file GitHub writer
6. UI and run-status surfacing